Python for Web Development

Lesson 4: Form Handling and Validation

Form Handling and Validation – Study Notes

Key Concepts

1. **Processing POST data** – retrieving and sanitizing user input from `request.form` (or `request.get_json`).
2. **WTForms integration** – defining form classes, field types, and built in validators.
3. **Custom validators** – writing reusable validation logic for complex rules.
4. **Flash messages** – providing user friendly feedback (success, error, info) across redirects.
5. **CSRF protection** – preventing cross site request forgery when handling forms.

Summary

In web applications, handling user submitted data safely and reliably is a core skill. When a client sends a **POST** request, the server must extract the payload, validate it, and react appropriately—either persisting the data or returning helpful error messages. Flask's `request` object gives direct access to raw form data, but manual checks quickly become error prone.

WTForms solves this by letting you declare a Python class that mirrors the HTML form. Each field (e.g., `StringField`, `PasswordField`, `SelectField`) can be coupled with built in validators such as `DataRequired`, `Length`, `Email`, or `EqualTo`. For business specific rules (e.g., "username must be unique"), you write **custom validator functions** that raise `ValidationError` when the rule fails.

After validation, the view either processes the data (e.g., creates a user, saves a record) or re renders the form with error messages. To inform the user about the outcome across a redirect, Flask's **flash** system stores short messages in the session and displays them on the next request. Coupled with proper **CSRF protection** (via `Flask WTF`'s `CSRFProtect`), this pattern yields a robust, user friendly form workflow.

Important Points

- **Accessing POST data**

```python
from flask import request
name = request.form.get('name')          # returns None if missing
age  = request.form['age']               # raises BadRequestKeyError if missing
```

- **Creating a WTForms form**

```python
from flask_wtf import FlaskForm
```

from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import DataRequired, Length, Email, EqualTo, ValidationError
```

• **Built  in validators**

- `DataRequired` – field cannot be empty (different from `InputRequired`).
- `Length(min, max)` – enforce string length.
- `Email` – simple email pattern check.
- `EqualTo('other_field')` – useful for password confirmation.

• **Custom validator example**

```python
def validate_username(self, field):
    if User.query.filter_by(username=field.data).first():
        raise ValidationError('Username already taken.')
```

• **Using the form in a view**

```python
@app.route('/register', methods=['GET', 'POST'])
def register():
    form = RegistrationForm()
    if form.validate_on_submit():          # POST + all validators pass
        # process data
        flash('Registration successful!', 'success')
        return redirect(url_for('login'))
    return render_template('register.html', form=form)
```

• **Flash messages**

- `flash(message, category)` – category can be `'success'`, `'error'`, `'warning'`, etc.
- In templates: `{% with messages = get_flashed_messages(with_categories=True) %}` loop to display.

• **CSRF token** – automatically added by `FlaskForm`; include `{{ form.hidden_tag() }}` in the template.

• **Error rendering** – `{{ form.field.errors }}` gives a list of error strings for that field.

Examples

Example 1 – Simple Contact Form

Python (forms.py)

```python
from flask_wtf import FlaskForm
from wtforms import StringField, TextAreaField, SubmitField
from wtforms.validators import DataRequired, Email, Length
class ContactForm(FlaskForm):
    name    = StringField('Name', validators=[DataRequired(), Length(max=50)])
```
```

```
    email   = StringField('Email', validators=[DataRequired(), Email()])
    message = TextAreaField('Message', validators=[DataRequired(), Length(min=10, max=500)])
    submit  = SubmitField('Send')
```

View (app.py)

```python
@app.route('/contact', methods=['GET', 'POST'])
def contact():
    form = ContactForm()
    if form.validate_on_submit():
        # pretend we send an email here
        flash('Your message has been sent!', 'success')
        return redirect(url_for('contact'))
    return render_template('contact.html', form=form)
```

Template (contact.html)

```html
{% extends "base.html" %}
{% block content %}
  <h2>Contact Us</h2>
  <form method="POST">
    {{ form.hidden_tag() }}
    <p>{{ form.name.label }} {{ form.name() }}</p>
    <p>{{ form.email.label }} {{ form.email() }}</p>
    <p>{{ form.message.label }} {{ form.message() }}</p>
    <p>{{ form.submit() }}</p>
  </form>
  {% with msgs = get_flashed_messages(with_categories=True) %}
    {% if msgs %}
      {% for category, msg in msgs %}
        <div class="flash {{ category }}">{{ msg }}</div>
      {% endfor %}
    {% endif %}
  {% endwith %}
{% endblock %}
```

Explanation: The form class declares fields and validators. In the view, `validate_on_submit()` checks both request method and validation. On success we flash a message and redirect to avoid duplicate submissions (Post Redirect Get pattern).

Example 2 – Registration with a Custom Username Validator

Form

```python
class RegistrationForm(FlaskForm):
    username = StringField('Username', validators=[DataRequired(), Length(min=4, max=25)])
    email    = StringField('Email', validators=[DataRequired(), Email()])
    password = PasswordField('Password', validators=[DataRequired(), Length(min=6)])
    confirm  = PasswordField('Confirm Password',
                             validators=[DataRequired(),
                                         EqualTo('password', message='Passwords must match')])
    submit   = SubmitField('Register')
    # custom validator method (named validate_<fieldname>)
    def validate_username(self, field):
        if User.query.filter_by(username=field.data).first():
            raise ValidationError('Username already exists. Choose another.')
```

View

```python
@app.route('/register', methods=['GET', 'POST'])
def register():
    form = RegistrationForm()
    if form.validate_on_submit():
        user = User(username=form.username.data,
                    email=form.email.data,
                    password=generate_password_hash(form.password.data))
        db.session.add(user)
        db.session.commit()
        flash('Account created! You can now log in.', 'success')
        return redirect(url_for('login'))
    return render_template('register.html', form=form)
```

Explanation: The `validate_username` method runs automatically after the built in validators. It queries the database; if a conflict is found, it raises `ValidationError`, which appears next to the field in the rendered form.

Quick Review

1. **What does `form.validate_on_submit()` check internally?**
2. **Name two built in WTForms validators and describe when you would use each.**
3. **How do you display flash messages in a Jinja2 template?**
4. **Explain the purpose of a CSRF token and how Flask WTF handles it.**
5. **Write a short custom validator that ensures a numeric field is a multiple of 5.**

Further Reading

• **Flask WTF Documentation** – deeper dive into form rendering, file uploads, and internationalization.

• **WTForms Advanced Validators** – `Regexp`, `AnyOf`, `NoneOf`, and creating reusable validator classes.

• **Post Redirect Get (PRG) Pattern** – why redirect after a successful POST improves UX and prevents duplicate submissions.

• **Security Best Practices** – CSRF, XSS mitigation, and safe handling of user generated content.

• **Testing Forms** – using Flask's test client to assert validation behavior and flash messages.